

Inter-Process Communication (IPC)

Inter-Process Communication (IPC)

IPC are the mechanisms provided by an operating system for processes to manage shared data while communicating with each other. Linux supports a number of IPC mechanisms, including

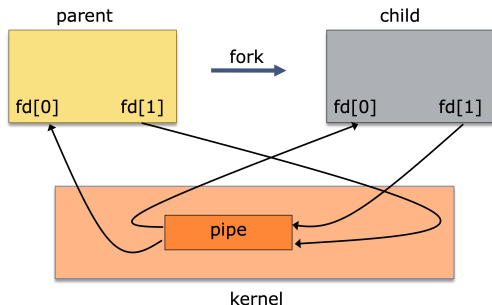
- Pipes
- First In, First Out (FIFO) – aka. “named pipes”.
- Signals
- Message Queues
- Shared Memory: type `ipcs` on your terminal shell to check out the detail.
- Semaphores
- Sockets (covered in Network)

IPC with PIPE

- Pipes are the oldest form of UNIX system IPC mechanism.
- it has have two limitations:
 - Historically, pipes are half duplex – data flows in only one direction. (Some system provide full duplex pipes.)
 - Pipes can be used only between processes that have a common ancestor (usually between parent and child).
- Two file descriptors are returned through the `filedes[2]` parameter.
 - `filedes[0]` is open for reading
 - `filedes[1]` is open for writing
- In a typical use, a process creates a pipe just before it forks one or more child processes.
- The pipe is then used for communication either between the parent and child processes, or between two sibling processes.

```
#include <unistd.h>
// Return 0 on success; -1 on error
int pipe (int filedes[2]);
```

Half-Duplex Pipe After a Fork



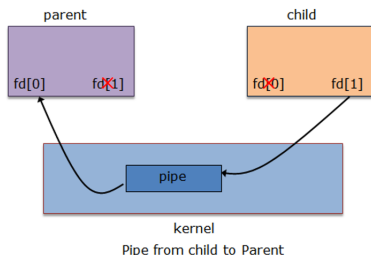
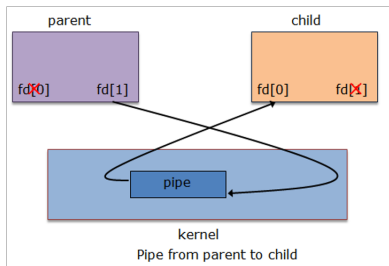
When one end of a pipe is closed, two rules apply.

- If we read from a pipe whose end has been closed, read returns 0 to indicate an end of file after all data has been read.
- If we write to a pipe whose read end has been closed, the signal SIGPIPE is generated. It can be ignored or handled.

Sending Data Between Parent and Child

Sending data from parent to child (left figure):

- parent closes `filedes[0]` (the read end of the pipe),
- child close `filedes[1]` (the write end of the pipe)



Sending data from child to parent (right figure):

- parent closes `filedes[1]` (the write end of the pipe),
- child close `filedes[0]` (the read end of the pipe)

Sending Data from Parent to Child - Code Example

```
#include <sys/types.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#define MAXLINE 256
void err_sys(char *str) {
    printf ("%s \n",str);
    exit (1);
}
int main() {
    int n, fd[2];
    pid_t pid;
    char line[MAXLINE];
    if (pipe(fd) < 0) /* creat a pipe */
        err_sys("pipe error");
    pid = fork();
    if (pid < 0) /*creat a child */
        err_sys("fork error");
    if (pid > 0) /* parent */ {
        close(fd[0]);
        write(fd[1], "May 12\n", 7); /*write to a pipe */
    }
    if(pid == 0) /* child */ {
        close(fd[1]);
        n = read(fd[0], line, MAXLINE); /*read from the pipe */
        write(STDOUT_FILENO, "Mother's day is ", 16);
        write(STDOUT_FILENO, line, n);
    }

    exit(0);
}
```

Sending Data from Parent to Child - Code Example 2

```
#include <sys/types.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
/* function prototypes */
void read_from_pipe (int);
void write_to_pipe (int);
void err_sys(char *);

int main (void) {
    pid_t pid;
    int fd[2];

    if (pipe (fd) < 0) /* Create the pipe. */
        err_sys("pipe failed");
    pid = fork();
    if (pid < 0) /*creat a child */
        err_sys("fork error");
    if (pid > 0) /* parent */ {
        close (fd[0]);
        write_to_pipe (fd[1]);
    }
    if(pid == 0) /*the child process */ {
        close (fd[1]);
        read_from_pipe (fd[0]);
    }
    exit (0);
}
```

Sending Data from Parent to Child - Code Example 2 (Cont')

```
/* Read characters from the pipe and echo them to stdout. */
void read_from_pipe (int fd) {

    FILE *stream;
    int c;
    stream = fdopen (fd, "r");
    while ((c = fgetc (stream)) != EOF) /* Read a character by character untile EOF */
        putchar (c);                  /* Write on standard output */

    fclose (stream);
}

/* Write some random text to the pipe. */
void write_to_pipe (int fd) {
    FILE *stream;

    stream = fdopen (fd, "w"); /* open a pipe for writing */
    fprintf (stream, "A butterfly talks to each flower\n");
    fprintf (stream, "A mockingbird sings every song\n");
    fclose (stream);
}

void err_sys(char *str) {
    printf ("%s \n",str);
    exit (1);
}
```


Sending Data from Child to Parent - Code Example

```
#include <sys/types.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#define MAXLINE 256
void err_sys(char *str) {
    printf ("%s \n",str);
    exit (1);
}
int main() {
    int n, fd[2];
    pid_t pid;
    char line[MAXLINE];
    if (pipe(fd) < 0) /* creat a pipe */
        err_sys("pipe error");
    if ( (pid = fork()) < 0) /*creat a child */
        err_sys("fork error");
    else if (pid > 0) /* parent */ {
        close(fd[1]);
        n = read(fd[0], line, MAXLINE); /*read from the pipe */
        write(STDOUT_FILENO, "hello baby ", 12);
        write(STDOUT_FILENO, line, n);
    }
    else /* child */ {
        close(fd[0]);
        write(fd[1], "hello mom\n", 10); /*write to a pipe */
    }
    exit(0);
}
```

popen() and pclose() function

These two function handle all work that we've been doing ourselves for using a pipe.

- Creating a pipe
- Forking a child
- Closing the unused ends of the pipe
- Executing a shell command, and
- Waiting for the command to terminate

```
#include <stdio.h>
// Return a file pointer if success, NULL on error
FILE *popen(const char *cmdstring, const char *mode);

//Return termination status on success, -1 on error
int pclose(FILE *fp);
```

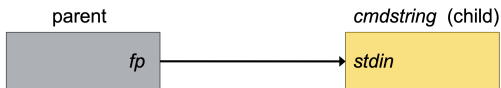
- The `popen()` function does `fork` and `exec` to execute the `cmdstring` and returns a standard I/O file pointer.
- If `mode = "r"`, the file pointer is connected to the standard output of the `cmdstring`.
- If `mode = "w"`, the file pointer is connected to the standard inputs of the `cmdstring`.

More on popen() Function

```
fp = popen (cmdstring, "r");
```



```
fp = popen (cmdstring, "w");
```



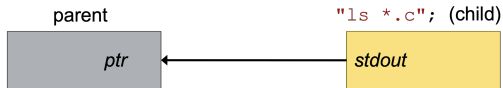
popen(), pclose() - Code Example

```
/* popen.c demonstrate the popen function. */
#include <stdio.h>
#include <stdlib.h>
int main( ) {
    char *cmd = "ls *.c";
    char buf[BUFSIZ]; /* BUFSIZ =1024 defined in stdio.h */
    FILE *ptr;
    if ((ptr = popen(cmd, "r")) != NULL)
        while (fgets(buf, BUFSIZ, ptr) != NULL)
            (void) printf("%s", buf);
    pclose(ptr);
    return 0;
}
```

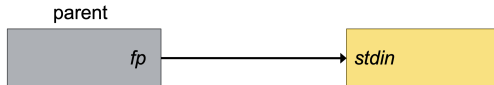
```
/* popen1.c: demonstrate popen with "w" */
#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
int main() {
    FILE *wfp;
    char buffer[BUFSIZ + 1];
    sprintf(buffer, "This is testing popen with w \n");
    wfp = popen("cat ", "w");
    if (wfp != NULL) {
        fwrite(buffer, sizeof(char), strlen(buffer), wfp);
        pclose(wfp);
        exit (0);
    }
    exit (1);
}
```

popen(), pclose() - Code Segment with Figure

```
char *cmd = "ls *.c";  
ptr = popen(cmd, "r");
```



```
wfp = popen("od -c ", "w");
```



FIFOs

Unlike Pipes, which can be used only between related processes with a common ancestor (usually parent and child), **FIFOs** (a.k.a. Named pipes) allow two unrelated processes to communicate with each other.

- FIFO is a type of file, creating a FIFO is similar to creating a file.
- Two unrelated processes can open a FIFO and begin communication.
- The specification of mode argument is the same as for the `open()` system call.
- The rule for user and group ownership of a FIFO are the same as for a file.
- Once we create a FIFO by using `mkfifo`, we can open it by using `open()`.

```
#include <sys/stat.h>
// return 0 if success; -1 on error
int mkfifo(const char *pathname, mode_t mode);
```

FIFO on Shell Example

FIFO - Code Example

```
#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>
#include <sys/types.h>
#include <sys/stat.h>

int main(int argc, char **argv) {
    int res = mkfifo(argv[1], 0666);
    if (res == 0)
        printf("FIFO created\n");
    exit(EXIT_SUCCESS);
}
```

FIFOs

- A FIFO supports blocked read and write operations by default:
 - A FIFO open for read-only blocks until some other process opens the FIFO for writing.
 - A FIFO open for write-only blocks until some other process opens the FIFO for reading.
- However, it is possible to make FIFOs support non-blocking operations by specifying the `O_NONBLOCK` flag while opening them. In this case:
 - An open for read-only returns immediately.
 - An open for write-only returns -1 with `errno` value set to `ENXIO` if no process has the FIFO open for reading.
- A FIFO must be opened either read-only or write-only. It must not be opened for read-write because it is half-duplex, that is, a one-way channel.

Four Valid Combinations

`open(const char *path, O_RDONLY);`

- will block until FIFO is opened for writing

`open(const char *path, O_RDONLY|O_NONBLOCK);`

- will succeed/return if FIFO is not opened for writing

`open(const char *path, O_WRONLY);`

- will block until FIFO is opened for reading

`open(const char *path, O_WRONLY|O_NONBLOCK);`

- will fail/return if FIFO is not opened for reading